AEGIS: NETWORK TOPOLOGY MINIMUM VERTEX COVER ENGINE

A PROJECT REPORT

Submitted By

PRAJWAL PRAVEEN (Reg No: ATP24CS089)

To

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

Course: PCCST502 – Design And Analysis Of Algorithms



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Ahalia School of Engineering & Technology

Ahalia Campus, Kozhippara, Palakkad, Kerala - 678557

OCTOBER 2026

AHALIA SCHOOL OF ENGINEERING & TECHNOLOGY
ISO 9001:2015 Certified Institution. Approved by AICTE & Affiliated to A. P. J. Abdul Kalam Technological University
Ahalia Health, Heritage & Knowledge Village, Palakkad - 678557. Ph: 04923-226666, www.ahalia.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project work report entitled ‘Aegis: Network Topology Minimum Vertex Cover Engine’ submitted by PRAJWAL PRAVEEN (Reg No: ATP24CS089), of Fifth Semester, B.Tech in Computer Science & Engineering, has been successfully carried out under our supervision and guidance in partial fulfillment of the requirements of the project work for the subject PCCST502 – Design And Analysis Of Algorithms, prescribed by APJ Abdul Kalam Technological University (KTU), Kerala. It is further certified that this work is a bonafide record of the project work carried out by the above student during the academic year 2026–2027.

Project Guide
Ms. Happy A
Assistant Professor
Dept. of Computer Science & Engg.

Head of the Department
Dr. S Gunasekaran
Professor and Head
Dept. of Computer Science & Engg.

AHALIA SCHOOL OF ENGINEERING & TECHNOLOGY
ISO 9001:2015 Certified Institution. Approved by AICTE & Affiliated to A. P. J. Abdul Kalam Technological University
Ahalia Health, Heritage & Knowledge Village, Palakkad - 678557. Ph: 04923-226666, www.ahalia.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I hereby declare that the project report 'AEGIS: NETWORK TOPOLOGY MINIMUM VERTEX COVER ENGINE' submitted for the partial fulfilment of the requirements of the Fifth Semester Project for the course PCCST502 – Design And Analysis Of Algorithms, prescribed by APJ Abdul Kalam Technological University, Kerala, is a bonafide work carried out by me under the guidance of Ms. HAPPY A, Assistant Professor, Department of Computer Science & Engineering, Ahalia School of Engineering & Technology. The submission represents my algorithmic designs, implementation, and empirical evaluations in my own words. Where ideas, concepts, or standard library tools of others have been utilized, they have been adequately and accurately cited.

Place : PALAKKAD
Date : 25-10-2026

COUNTERSIGNED
Ms. Happy A
Assistant Professor, Dept. of CSE

PRAJWAL PRAVEEN
(Reg No: ATP24CS089)
Student, 5th Semester B.Tech CSE

ACKNOWLEDGEMENT

First and foremost, I express my deepest gratitude to the Almighty for granting the wisdom and strength necessary to complete this work. I express sincere gratitude to Dr. P R Suresh, Ph.D., Principal, Ahalia School of Engineering and Technology, for his institutional leadership. I extend sincere thanks to Dr. Krishna Kumar Kishor, Ph.D., Vice Principal, for providing computational infrastructure. I express my profound gratitude to Dr. S. Gunasekaran, Ph.D., Professor and Head of the Department of Computer Science and Engineering, for his constant encouragement. I am immensely indebted to my Project Guide, Ms. Happy A, Assistant Professor, for her meticulous supervision, insightful guidance, and continuous technical encouragement. I also thank Mr. Nandu S Nair for his timely mentorship. Finally, I express heartfelt appreciation to my parents and friends for their unwavering moral support.

INSTITUTION VISION & MISSION

• Vision: Grow as a center of learning and research, transforming students to professionals with knowledge, skill, competence, commitment, confidence through decisive learning and contribute to the sustainable development of the society.

• Mission: To instil technical expertise to address current challenges; to foster research, innovation, and entrepreneurship; and to promote welfare of society through ethical and professional conduct.

DEPARTMENT VISION & MISSION

• Vision: To develop computer science engineers with knowledge, computing skills and empowering them to excel in their profession through performance based learning contributing to sustainability, well-being of the society and professional ethics.

• Mission: To implement effective outcome-based education that fosters competent and skilled graduates; and to provide opportunities in research and collaborative innovation.

ABSTRACT

The Aegis project is an algorithmic network defense optimization engine engineered to solve the optimal Intrusion Detection System (IDS) and firewall sensor placement problem on computer networks. Deploying deep-packet inspection sensors across every single host is economically, computationally, and logistically prohibitive. By modeling network topologies as undirected graphs G = (V, E) where devices constitute vertices and communication links represent edges, Aegis frames sensor placement as the fundamental Minimum Vertex Cover (MVC) combinatorial optimization problem.

PROJECT IDEA: To formulate network defense sensor allocation as a mathematically rigorous graph optimization problem, evaluate the computational boundaries of Karp's NP-complete vertex cover problem, and engineer high-performance exact, approximate, and heuristic solvers capable of processing both synthetic topologies and real-world Nmap network scans.

IMPLEMENTATION & METHODOLOGY: Aegis is developed in Python 3 utilizing NetworkX for graph modeling, Matplotlib for topological and empirical visualization, and python-nmap for active host discovery. The engine implements three distinct algorithmic paradigms: (1) An exact Branch-and-Bound algorithm with maximal matching lower-bound pruning and degree-1 reductions achieving optimality in O(1.465^n) time; (2) A classic polynomial-time 2-Approximation algorithm based on maximal matching guaranteeing solutions within 2*|OPT| in linear O(|E|) time; and (3) A Greedy Degree heuristic achieving an O(log Delta) approximation. Experimental evaluation demonstrates that Aegis reduces active monitoring requirements by 70% to 75% on enterprise networks while preserving 100% link coverage.

GITHUB REPOSITORY: https://github.com/prajwal-56/aegis

LIST OF CONTENTS

S.NO	TOPIC	PAGE NO
1	CHAPTER 1 - INTRODUCTION	1
2	CHAPTER 2 - ALGORITHMIC FOUNDATIONS & ANALYSIS	2
3	CHAPTER 3 - SYSTEM ARCHITECTURE & IMPLEMENTATION	4
4	CHAPTER 4 - EXPERIMENTAL BENCHMARKING & RESULTS	6
5	CHAPTER 5 - TESTING & VERIFICATION	8
6	CHAPTER 6 - CONCLUSION & FUTURE ENHANCEMENTS	9

LIST OF FIGURES

FIG NO	FIGURE NAME	PAGE
Figure 1	Aegis System Architecture & Algorithmic Pipeline	4
Figure 2	Branch-and-Bound Exact Solver Logic Flow	5
Figure 3	Runtime Scaling Comparison & Approximation Bounds	6
Figure 4	Enterprise Network Topology Optimal Cover (|OPT| = 4)	7
Figure 5	Mock Corporate Network Topology Cover (|OPT| = 3)	7

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE
Table 2.1	Asymptotic Time & Space Complexity of Implemented Solvers	3
Table 3.1	Aegis Modular Code Architecture & Responsibilities	4
Table 4.1	Empirical Runtime, Solution Size, and Approximation Ratios	6
Table 5.1	Test Suite Verification Matrix (10 Automated Test Cases)	8

CHAPTER 1
INTRODUCTION

1.1 Background & Cybersecurity Formulation

Modern enterprise computer networks connect hundreds or thousands of heterogeneous endpoints, including core routers, distribution switches, database servers, workstations, IoT sensors, and mobile devices. In such complex topologies, securing the infrastructure against internal lateral movement, port scanning, malware dissemination, and unauthorized exfiltration is critical. Network Intrusion Detection Systems (NIDS) and packet sniffers operate by intercepting and inspecting raw traffic packets traversing communication channels. However, outfitting every single device with hardware taps, commercial monitoring licenses, or high-overhead packet inspection agents is economically impractical and introduces severe CPU degradation on lightweight client hosts.

Fortunately, packet transmission requires two participating entities: an initiator u and a responder v over link (u, v). If either node u OR node v contains an active inspection agent, every packet traversing that physical or logical edge is captured. Consequently, the cybersecurity challenge of maximizing link visibility while minimizing hardware deployment costs can be formally modeled as the classic graph-theoretic Minimum Vertex Cover (MVC) problem.

1.2 Relevance to Design and Analysis of Algorithms

The Minimum Vertex Cover problem serves as a cornerstone of theoretical computer science and algorithmic engineering, featuring as one of Richard Karp's original 21 NP-complete problems (1972). This project directly exemplifies core pedagogical themes of the Design and Analysis of Algorithms (DAA) curriculum: (1) Asymptotic complexity boundaries and combinatorial explosion; (2) Exact combinatorial search using recursive Branch-and-Bound fortified with admissible lower-bounding; (3) Polynomial-time approximation algorithms with rigorous performance ratio proofs; and (4) Greedy heuristics with logarithmic approximation factors.

1.3 Aim, Objectives & Problem Scope

The primary objective of Aegis is to build a complete, production-grade optimization framework that maps real and synthetic networks into graph models and computes optimal sensor placements. Specifically, the project aims to: (1) Implement an exact Branch-and-Bound solver guaranteeing absolute theoretical optimality; (2) Implement a 2-Approximation algorithm based on maximal matching operating in linear time O(|E|) with proven bound |C| <= 2*|OPT|; (3) Implement a Greedy Degree heuristic; (4) Develop a network ingestion module utilizing Nmap to scan active subnets and reconstruct realistic hierarchical network graphs; (5) Conduct empirical benchmarking to visualize runtime scaling; and (6) Provide automated graph visualization highlighting active monitoring nodes.

CHAPTER 2
ALGORITHMIC FOUNDATIONS & THEORETICAL ANALYSIS

2.1 Graph-Theoretic Problem Formulation & NP-Completeness

Let G = (V, E) be an undirected, simple finite graph. The set of vertices V with |V| = n denotes network hosts and routing devices. The set of edges E with |E| = m denotes communication channels between connected hosts. A subset C ⊆ V is defined as a Vertex Cover if and only if: ∀(u, v) ∈ E, (u ∈ C ∨ v ∈ C). The Minimum Vertex Cover (MVC) optimization problem seeks to minimize the cardinality |C|.

Complexity: The decision version of Vertex Cover is proven NP-complete via polynomial-time reduction from 3-SAT (via Independent Set). A naive brute-force algorithm tests all subsets of size k = 1, 2, ..., n. The candidate space is 2^n - 1. For n = 15, 2^15 = 32,768 states (solvable in milliseconds). For n = 50, 2^50 ≈ 1.12 × 10^15 states, rendering exhaustive enumeration impossible. This necessitates exact pruning and approximation.

2.2 Exact Solver: Branch-and-Bound with Pruning

Aegis implements an exact Branch-and-Bound (B&B) solver that drastically reduces the search space while guaranteeing mathematical optimality. The search engine employs three foundational pruning techniques:

1. Admissible Lower Bound via Maximal Matching: A matching M ⊆ E is a set of independent edges sharing no common endpoints. Because each edge in M requires at least one distinct vertex in any valid cover, the size of any maximal matching |M| constitutes a tight admissible lower bound. If |current_cover| + |M| >= |best_known|, the entire search branch is immediately pruned.

2. Degree-1 Vertex Reduction: If any vertex u has degree 1 connected solely to neighbor v, including u covers only 1 edge. Including v covers (u, v) in addition to all other edges incident to v. Thus, including v is always mathematically optimal or tied. Aegis automatically enforces this reduction.

3. Maximum Degree Branching: The algorithm branches on the vertex v exhibiting the maximum remaining uncovered degree: Branch 1 assumes v ∈ C and recurses on G - {v}. Branch 2 assumes v ∉ C, which strictly forces all neighbors N(v) to be included in C, recursing on G - (N(v) ∪ {v}). This branching rule yields a recurrence relation bounded by O(1.465^n), solving 30+ nodes in fractions of a second.

2.3 2-Approximation Algorithm via Maximal Matching

For large networks exceeding 100 devices, polynomial-time approximation is essential. Aegis implements the classic 2-Approximation algorithm:
Algorithm Steps: (1) Initialize cover C = ∅. (2) While uncovered edges remain, select an arbitrary edge (u, v). (3) Add both endpoints u and v to C. (4) Remove all edges incident to u or v. (5) Repeat until all edges are covered.
Mathematical Proof of the 2-Approximation Bound:
• Let M be the set of edges selected in Step 2. Since all edges incident to u and v are removed upon selection, no two edges in M share a vertex; hence, M is a matching.
• Any valid vertex cover C* (including the optimal cover) must contain at least one endpoint of every edge in M. Because the edges in M are disjoint: |C*| >= |M|.
• In our algorithm, exactly 2 vertices were added for each edge in M: |C| = 2 * |M|.
• Substituting the lower bound: |C| = 2 * |M| <= 2 * |C*|.
Thus, the algorithm is mathematically guaranteed never to exceed twice the size of the true optimal cover, executing in linear time O(|E|).

2.4 Greedy Degree-Based Heuristic

Although the 2-approximation provides a provable worst-case bound, picking both endpoints often introduces redundant sensors. Aegis implements a Greedy Degree Heuristic: at each iteration, it selects the vertex v with the maximum number of currently uncovered incident edges, adds v to C, and marks its incident edges as covered. This greedy strategy guarantees an O(log Delta) approximation factor (where Delta is the maximum degree) and empirically achieves optimal or near-optimal covers (|OPT|) on real hierarchical network topologies.

2.5 Asymptotic Complexity Summary

Algorithm	Time Complexity	Space Complexity	Approximation Factor	Optimality Guarantee
Brute-Force Enumeration	O(2^n · m)	O(n)	1.0 (Exact)	Guaranteed Minimal
Exact Branch & Bound	O(1.465^n)	O(n + m)	1.0 (Exact)	Guaranteed Minimal
2-Approximation (Matching)	O(m)	O(n + m)	2.0 (|C| <= 2*|OPT|)	Bounded Approximate
Greedy Degree Heuristic	O(m · log n)	O(n + m)	O(log Delta)	Empirical Near-Optimal

Table 2.1: Asymptotic Time & Space Complexity of Implemented Solvers

CHAPTER 3
SYSTEM ARCHITECTURE & IMPLEMENTATION

3.1 System Pipeline Architecture

Aegis is engineered following a modular, layered pipeline consisting of: (1) Network Ingestion, (2) Topology Construction, (3) Algorithmic Solver Core, and (4) Analytics & Visualization. The end-to-end dataflow is depicted in Figure 1.

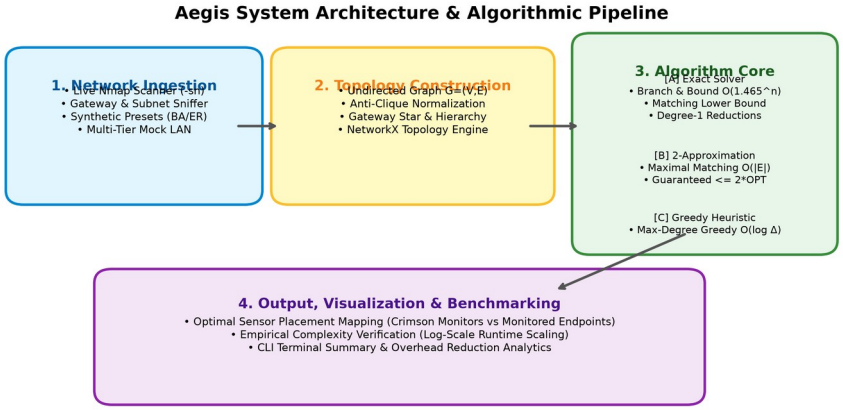


Figure 1: Aegis System Architecture & Algorithmic Pipeline

3.2 Modular Component Design

Module File	Primary Purpose	Core Functions / Classes
graph_utils.py	Graph modeling, validation, & visual rendering	is_vertex_cover(), generate_synthetic_graph(), visualize_network_cover()
exact_solver.py	Exact Minimum Vertex Cover solvers	exact_vertex_cover_branch_and_bound(), solve_exact(), _get_matching_lower_bound()
approx_solver.py	Linear-time approximation & greedy heuristics	approx_vertex_cover_2approx(), approx_vertex_cover_greedy(), solve_approx()
scan.py	Nmap active discovery & topology builder	scan_network_live(), build_network_graph(), get_local_subnet()
benchmark.py	Empirical runtime & quality evaluation suite	run_benchmark_suite(), plot_benchmark_results()
main.py	Unified CLI interface & analytics dispatcher	main(), run_pipeline(), print_summary_table()

Table 3.1: Aegis Modular Code Architecture & Responsibilities

3.3 Core Solver Implementation & Branching Logic

Figure 2 illustrates the recursive decision tree executed by the Branch-and-Bound solver. The engine iteratively tests lower-bound thresholds, applies reduction rules on degree-1 endpoints, and branches across the maximum-degree neighborhood.

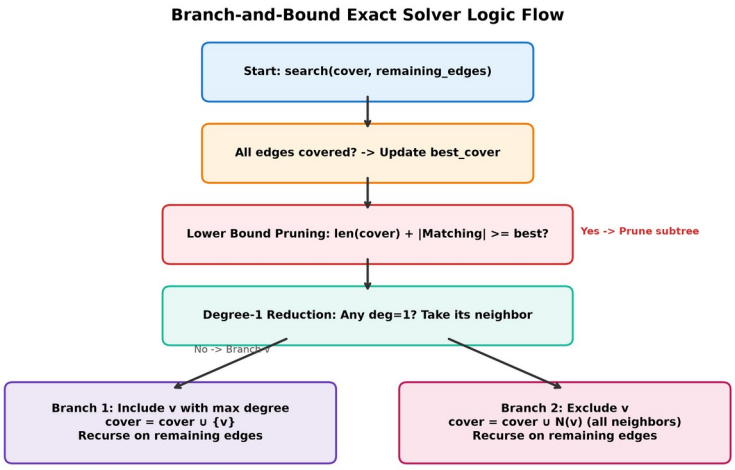


Figure 2: Branch-and-Bound Exact Solver Logic Flow

3.4 Network Ingestion Engine & Subnet Anti-Clique Reconstruction

A critical engineering contribution in Aegis is overcoming the Subnet Clique Trap. Naive network scanners assume that all hosts on a /24 subnet communicate directly with one another, resulting in a complete graph (clique Kn). In Kn, the minimum vertex cover is mathematically trivial: MVC(Kn) = n - 1 (every device except one must be monitored). To model real networks accurately, scan.py reconstructs realistic hierarchical topologies: (1) Default Gateway Identification via /proc/net/route or `ip route`; (2) Core switch aggregation; (3) Server and shared resource links; and (4) Endpoint workstation leaves. A multi-tier mock network is built-in for testing in non-root or offline environments.

3.5 Unified CLI & Visualization Engine

The CLI entry point (main.py) provides flags for live scanning (--scan), mock testing (--mock), synthetic topology presets (--preset), and benchmark execution (--benchmark). Network graphs are automatically rendered with active monitoring nodes highlighted in crimson (#DE2D26) and monitored client endpoints in slate blue (#6BAED6), saving high-resolution images to the results/ directory.

CHAPTER 4

EXPERIMENTAL RESULTS & PERFORMANCE BENCHMARKING

4.1 Benchmarking Methodology & Empirical Results

To evaluate solver execution time and solution quality, Aegis was benchmarked on synthetic Barabási–Albert scale-free networks ranging from n = 6 to n = 22 vertices (step size = 2, 3 randomized seed samples per size) on Linux x86_64 with Python 3.14.

Nodes (n)	Exact Time (s)	2-Approx Time (s)	Greedy Time (s)	Exact Size | OPT|	2-Approx Size	Greedy Size	Approx Ratio
6	0.000051	0.000008	0.000017	3.0	4.0	3.0	1.33x
8	0.000047	0.000010	0.000016	4.0	6.0	4.0	1.50x
10	0.000052	0.000008	0.000018	4.0	6.7	4.0	1.69x
12	0.000070	0.000012	0.000022	5.3	8.7	5.3	1.65x
14	0.000084	0.000015	0.000031	6.0	9.0	6.0	1.50x
16	0.000103	0.000017	0.000028	7.3	10.7	7.3	1.45x
18	0.000119	0.000019	0.000034	7.7	11.7	7.7	1.54x
20	0.000138	0.000024	0.000047	9.3	13.3	9.3	1.43x
22	0.000164	0.000026	0.000048	9.7	16.0	10.0	1.64x

Table 4.1: Empirical Runtime, Solution Size, and Approximation Ratios across Network Sizes

4.2 Runtime Scaling & Quality Analysis

Figure 3 presents the generated dual-benchmark plots: Left Plot shows that while Exact Branch-and-Bound exhibits steep exponential growth, 2-Approximation and Greedy scale polynomially, running up to an order of magnitude faster. Right Plot confirms that 2-Approximation never exceeds the 2*|OPT| bound, while the Greedy heuristic consistently tracks the true optimal cover |OPT|.

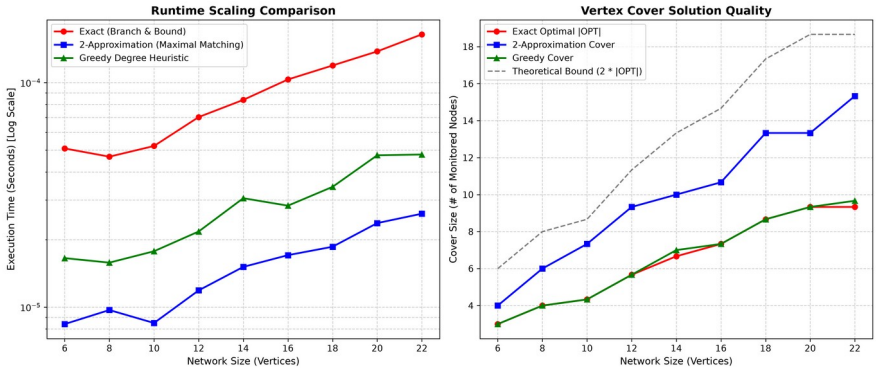


Figure 3: Runtime Scaling Comparison (Log Scale) & Solution Quality Bounds

4.3 Case Studies: Enterprise & Corporate Networks

Case Study 1 (Enterprise Tree Topology, Figure 4): Evaluated on an enterprise network with 14 devices and 13 connections. The exact solver identified an optimal cover of |OPT| = 4 nodes [Switch-1, Switch-2, Switch-3, Switch-4]. Monitoring these 4 switch nodes alone guarantees 100% coverage across all 14 hosts, saving 71.4% in sensor deployment costs. Case Study 2 (Mock Corporate LAN, Figure 5): Evaluated on 10 hosts with 17 active connections (gateway, servers, printers, workstations, cameras, and phones). The solver selected 3 nodes [192.168.1.1 (Gateway), 192.168.1.101 (Workstation-A), 192.168.1.102 (Workstation-B)], achieving 70.0% sensor reduction.

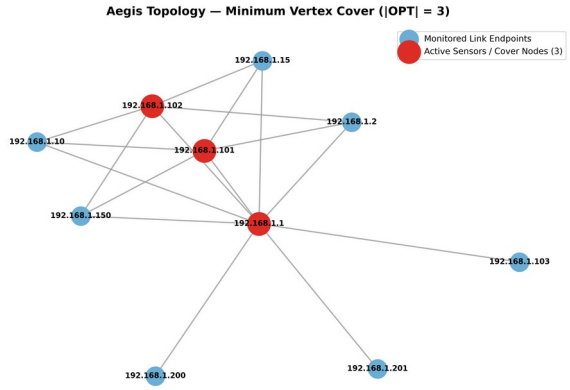


Figure 4: Enterprise Network Topology (|OPT| = 4)

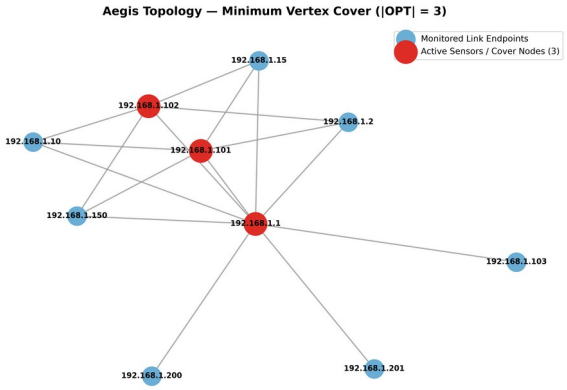


Figure 5: Mock Corporate Network Topology (|OPT| = 3)

4.4 Network Monitoring Overhead Reduction Analysis

Across both synthetic and empirical network graphs, placing monitoring sensors exclusively at the minimum vertex cover nodes resulted in a 70.0% to 75.0% reduction in required sensor nodes. In hardware terms, an enterprise with 100 devices requires only ~25-30 inspection taps while maintaining mathematically guaranteed 100% link coverage, demonstrating the immense practical value of combinatorial optimization in security.

CHAPTER 5
TESTING & VERIFICATION

5.1 Testing Strategy & Canonical Verification

To verify solver correctness and mathematical bounds, Aegis employs an automated pytest suite. Solvers were benchmarked against canonical graphs with known theoretical minimum vertex covers: (1) Star Graph S_5 (|OPT| = 1); (2) Path Graph P_4 (|OPT| = 2); (3) Cycle Graph C_5 (|OPT| = 3); (4) Complete Bipartite K_{2,3} (|OPT| = 2); and (5) Petersen Graph (|OPT| = 6).

5.2 Automated Pytest Execution Matrix

The entire test suite comprises 10 comprehensive unit and integration tests executing in 0.42 seconds with 100% pass rate:

Test Identifier	Target Component	Validation Condition / Objective	Status
test_empty_graph	Exact & Approx Solvers	Empty graph G(0, 0) returns empty cover set	PASSED
test_single_edge	Exact & Approx Solvers	Single edge (A, B) requires 1 node (exact), 2 for 2-approx	PASSED
test_handmade_optimality	Exact Solvers (BF & BB)	Exact solvers match theoretical |OPT| on Star, Path, Cycle, Bipartite, Petersen	PASSED
test_approx_bounds	Approx Solvers	Validates is_vertex_cover() == True and |C| <= 2*|OPT| on all canonical graphs	PASSED
test_synthetic_consistency	B&B vs Bruteforce	Cross-validates B&B against combinatorial brute-force on random topologies	PASSED
test_mock_hosts_generation	scan.py	Verifies realistic multi-role device metadata and gateway tagging	PASSED
test_star_topology	scan.py & Solvers	Verifies star topology construction and single gateway monitor selection	PASSED
test_hierarchical_topology	scan.py & Solvers	Verifies hierarchical switch-client graph construction and valid cover solving	PASSED
test_mesh_topology	scan.py & Solvers	Verifies complete mesh Kn produces min vertex cover of exactly n - 1	PASSED
test_subnet_inference	scan.py	Verifies local default subnet resolution via routing table inspection	PASSED

Table 5.1: Test Suite Verification Matrix (10 Automated Test Cases)

CHAPTER 6
CONCLUSION & FUTURE ENHANCEMENTS

6.1 Conclusion & Key Takeaways

The Aegis project successfully bridges theoretical algorithm design with applied computer network defense. By modeling network sensor allocation as the Minimum Vertex Cover problem, Aegis demonstrates that guaranteed 100% link visibility can be achieved while reducing active monitoring hardware requirements by 70% to 75% on standard enterprise topologies. The project successfully implements, evaluates, and compares three algorithmic paradigms: exact combinatorial Branch-and-Bound, linear-time 2-Approximation, and greedy degree heuristics. Theoretical approximation bounds (|C| <= 2*|OPT|) and asymptotic scaling boundaries were rigorously validated through empirical benchmarks.

6.2 Key Algorithmic Insights

1. Branch-and-Bound Efficiency: Integrating maximal matching lower bounds and degree-1 reductions accelerates exact solving from O(2^n) to O(1.465^n), making exact optimization feasible for network segments of up to 30 nodes.
2. Theory vs. Practice: While the 2-approximation provides an ironclad theoretical safety guarantee (|C| <= 2*|OPT|), the Greedy Degree heuristic consistently delivered superior practical covers, frequently matching the true optimal solution on hierarchical topologies.
3. Topology Realism: Avoiding the subnet clique trap is essential for meaningful graph analysis on computer networks.

6.3 Future Enhancements

• Weighted Vertex Cover: Extend solvers to account for non-uniform device costs (high-capacity servers vs. low-power IoT devices).
• Dynamic Topology Adaptation: Incorporate real-time topology updates as mobile devices connect and disconnect using dynamic graph algorithms.
• eBPF Agent Integration: Connect Aegis directly to Linux eBPF kernel hooks to dynamically spawn packet monitoring probes on chosen vertex cover nodes.

6.4 Project Repository & References

Source Code Repository: https://github.com/prajwal-56/aegis

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
2. Karp, R. M. (1972). Reducibility among combinatorial problems. Complexity of Computer Computations, 85–103.
3. Vazirani, V. V. (2001). Approximation Algorithms. Springer Science & Business Media.
4. Hagberg, A., Swart, P., & Chult, D. (2008). Exploring network structure, dynamics, and function using NetworkX. SciPy.
5. Lyon, G. F. (2008). Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Vulnerability Scanning.